

DealFlow360 — Interface Control Document (ICD)

Contract between Team A (Backend + Frontend / Core Platform) and Team B (Smart Layer)

Purpose of this document: eliminate every possible "who builds this / how does this connect" conversation during the hackathon. If a question isn't answered here, it gets added here — not solved verbally and forgotten.

1. Ownership Matrix

Rule: **one table, one owner**. No module writes to a table it doesn't own — even the smart layer.

Data / Logic	Owner	Reasoning
<code>User</code> , <code>Customer</code> , <code>Product</code> , <code>PriceList</code> , <code>Warehouse</code> , <code>Stock</code>	Team A	Master/reference data — config-time, no intelligence needed
<code>Quotation</code> , <code>QuotationLine</code>	Team A	Core transactional entity; smart layer only <i>reads a snapshot via event</i> , never writes
<code>DiscountTier</code> , <code>CategoryDiscountLimit</code> , <code>ApprovalChainRule</code>	Team A	Configuration rules — smart layer <i>consumes</i> these rules but doesn't own them
Blended risk score calculation logic	Team B	This is intelligence, not storage — Team B computes it, Team A stores the result field on <code>Quotation</code>
<code>Approval</code> records	Team A	Team A creates/updates these when it <i>receives</i> a risk-score event; Team B never writes here
Upsell ranking logic	Team B	Pure computation over co-purchase + margin data passed to it
<code>UpsellRule</code> (config: pairings, promo tags)	Team A (admin CRUD)	Config data; Team B reads it as input, doesn't own it
<code>FulfillmentSplit</code>	Team A	Team A owns the record; basic split algorithm also starts in Team A (see §5)
Smarter split optimization (cost/shipment minimization upgrade)	Team B (post-MVP)	Optional enhancement layer — see Extension Contract §6
<code>DealHealthFlag</code>	Team B	Team B owns this table fully — detection AND storage, since nothing else writes here
<code>Subscription</code> , <code>Invoice</code> , <code>Payment</code>	Team A	Pure transactional/financial logic, no intelligence required
<code>AuditLog</code>	Team A	Written by an internal listener that subscribes to almost every event

Reasoning for the whole matrix: every row that says “Team B computes, Team A stores” exists so that if Team B's logic breaks or is late, Team A's core flow still works with a safe default (see §4 Fallback Contract). Intelligence is never a single point of failure for the core transaction.

2. The Golden Rule: No Cross-Team Direct DB Reads

Previous draft of this plan allowed the smart layer to read core tables directly. This ICD supersedes that.

Why the change: if Team B's risk engine directly queries the `Quotation` table, then the moment Team A renames a column or changes a join, Team B's code breaks with no warning — and in a hackathon with 2-day sprints, nobody has time to debug a silent cross-team break the night before a demo.

The fix: every event Team A publishes carries a **complete, self-contained payload** — everything Team B needs to do its job is *in the event*, not fetched separately. Team B never runs a SQL query against Team A's tables, and Team A never runs a query against Team B's tables. The event **is** the interface.

3. Event Catalog (the actual contract — lock this before Sprint 2)

Format: `EventName` — **published by** — **consumed by** — payload shape — what happens next.

3.1 `QuotationUpdated`

Published by: Team A, `quotation.service.ts`, on every line add/edit/discount change

Consumed by: Team B, `discount-risk-engine`, `upsell-engine`

Payload:

```

{
  "eventVersion": 1,
  "quotationId": "uuid",
  "customerId": "uuid",
  "customerTier": "Bronze | Silver | Gold",
  "salesRepId": "uuid",
  "lines": [
    {
      "lineId": "uuid",
      "productId": "uuid",
      "category": "Hardware | Services | Subscriptions",
      "qty": 2,
      "unitPrice": 1200.00,
      "discountPct": 12,
      "categoryMaxDiscountPct": 15
    }
  ],
  "timestamp": "ISO8601"
}
```

Why this shape: `categoryMaxDiscountPct` is included *in the event*, not looked up separately by Team B — Team B should never need to call Team A's discount-config table to know the limit. Team A resolves the

limit and hands it over already attached to the line.

3.2 RiskScoreComputed (response event)

Published by: Team B, discount-risk-engine

Consumed by: Team A, quotation.service.ts (stores score), approval.service.ts (decides routing)

Payload:

```
{
  "eventVersion": 1,
  "quotationId": "uuid",
  "blendedRiskScore": 0.62,
  "requiresApproval": true,
  "requiresFinance": false,
  "flaggedLines": ["lineId1", "lineId2"],
  "reason": "Setup Service line exceeds category limit by 8 points",
  "computedAt": "ISO8601"
}
```

Why requiresApproval / requiresFinance are booleans, not left for Team A to re-derive: Team B owns the *decision logic*, Team A just acts on the result. If the threshold rule changes, only Team B's code changes — Team A's approval service doesn't need to know the rule, only the verdict.

3.3 ApprovalRequired

Published by: Team A, approval.service.ts, after receiving RiskScoreComputed with requiresApproval: true

Consumed by: Team A internally (creates Approval record); also consumed by audit-log

Same team on both ends — included here for completeness of the flow.

3.4 QuotationApproved

Published by: Team A, approval.service.ts

Consumed by: Team A fulfillment.service.ts; Team B fulfillment-optimizer (post-MVP only)

Payload:

```
{
  "eventVersion": 1,
  "quotationId": "uuid",
  "approvedLines": [{ "productId": "uuid", "qtyOrdered": 2 }],
  "approvedAt": "ISO8601"
}
```

3.5 SplitSuggested

Published by: Team A (basic) or Team B (optimized version, if built) — **both publish the same event shape**, so the frontend doesn't care which one produced it

Payload:

json

```
{
  "eventVersion": 1,
  "quotationId": "uuid",
  "splits": [
    { "warehouseId": "uuid", "warehouseName": "Main Warehouse", "qty": 18, "estCost": 40 }
  ],
  "generatedBy": "core-rule | smart-optimizer"
}
```

Why generatedBy exists: lets you demo "basic version" vs "AI-optimized version" side by side to mentors — a genuine selling point for a hackathon judge.

3.6 UpsellSuggestionsRequested → UpsellSuggestionsReady

Same request/response pattern as 3.1/3.2, payload includes candidate products + margin data; response includes ranked list with marginDelta per suggestion.

3.7 NegotiationSubmitted (from Customer Portal)

Published by: Team A, customer-portal/portal.service.ts

Consumed by: Team A quotation.service.ts (applies requested change → re-triggers QuotationUpdated, restarting the flow from 3.1)

No Team B involvement needed here — it's a re-entry into the existing pipeline, which is why the pipeline being event-driven matters.

3.8 DealHealthFlagRaised

Published by: Team B, deal-health-engine (on a scheduled job + on QuotationUpdated)

Consumed by: Team A frontend (reads via REST, not the event directly — see §4)

Payload:

json

```
{
  "eventVersion": 1,
  "quotationId": "uuid",
  "flagType": "stalled | discount_anomaly | delivery_slippage",
  "severity": "low | medium | high",
  "detail": "Inactive for 9 days, avg rep response time is 2 days",
  "detectedAt": "ISO8601"
}
```

4. Read Contract (how the frontend gets Team B's output)

Team B's results are **never** fetched by the frontend directly from an event stream — the frontend only talks REST to Team A's API layer. Team B exposes small **read-only REST endpoints** that Team A's API gateway proxies to:

Endpoint	Owner	Returns
<code>GET /api/risk-score/:quotationId</code>	Team B	last computed score + reasoning
<code>GET /api/upsell-suggestions/:quotationId</code>	Team B	ranked suggestion list
<code>GET /api/deal-health-flags</code>	Team B	all open flags, filterable by severity

Why this two-tier setup (event for write path, REST for read path): events are fire-and-forget and async — great for triggering work, bad for a frontend that needs an immediate answer on page load. REST-on-demand solves the read side without polling the event bus.

5. Fallback / Failure Contract (critical — do not skip)

Rule: the core transaction must never be blocked by the smart layer being slow, broken, or not-yet-built.

Scenario	Fallback behavior	Reasoning
<code>discount-risk-engine</code> doesn't respond within 2 seconds	Team A treats it as <code>requiresApproval: true</code> (fail-safe, not fail-open)	Never let a missing risk score silently approve an over-discounted deal
<code>upsell-engine</code> fails/times out	Panel shows "No suggestions available" — quote builder works normally	Upsell is a nice-to-have, must never block the core quote flow
<code>deal-health-engine</code> job fails	Dashboard simply shows stale/no fails, logs an error	Reporting failure ≠ transactional failure, acceptable degradation
<code>fulfillment-optimizer</code> (post-MVP) not built yet	<code>SplitSuggested</code> comes only from Team A's basic rule (<code>generatedBy: "core-rule"</code>)	This is <i>why</i> the event shape is shared — zero frontend changes needed when Team B's version is added later

6. Extension Contract (how new smart features get added later — the "no code change" proof)

To add a brand-new smart feature (e.g. churn prediction) after the hackathon:

Write a new listener in `smart-layer/<new-feature>/` that subscribes to an existing event (e.g. `QuotationUpdated`)

Publish a new response event or write to a new table you own

Optionally expose one new `GET` endpoint

Zero lines change in any Team A module.

This is the concrete proof-point for your mentor's "connectable, extendable without touching existing code" requirement — point to this section directly.

7. Data Format Standards (avoid the silent bugs)

Field type	Standard	Reasoning
IDs	UUID string	Avoids collision across seed/test data generated independently by both teams
Timestamps	ISO8601 string, UTC	No timezone ambiguity between two teams' machines

Field type	Standard	Reasoning
Discount / percentages	Plain number, e.g. <code>12</code> means 12% (not <code>0.12</code>)	One team using fractions and the other using whole numbers is the single most common integration bug — pick one, document it, done
Currency	Number with 2 decimal places, single currency (multi-currency is out of MVP scope)	Matches brief explicitly marking multi-currency as bonus
Event envelope	Every payload includes <code>eventVersion: 1</code>	If a payload shape must change mid-hackathon, bump the version rather than silently breaking the other team's listener

8. Sign-off Checklist (do this in a 15-minute sync before Sprint 2 starts)

- ☐ Both teams have read §3 Event Catalog and agree on every payload shape
- ☐ Both teams agree on §7 format standards (especially: percentages as whole numbers, not fractions)
- ☐ Team A has stubbed all events as no-ops so Team B can start listening even before real logic exists
- ☐ Team B has stubbed all read endpoints returning mock data so Team A's frontend can build against them immediately
- ☐ Fallback behaviors in §5 are implemented **first**, before the real smart logic — this means the demo never breaks even if Team B's algorithm is unfinished